



INDIAN INSTITUTE OF
INFORMATION
TECHNOLOGY

Cloud Data Center and AI Services

Dr. Animesh Chaturvedi

Assistant Professor: **IIIT Dharwad**

Young Researcher: **Heidelberg Laureate Forum**
and **Pingala Interaction in Computing**

Young Scientist: **Lindau Nobel Laureate Meetings**

Postdoc: **King's College London** & **The Alan Turing Institute**

PhD: **IIT Indore** MTech: **IIITDM Jabalpur**



The Edge-to-Cloud AI Continuum

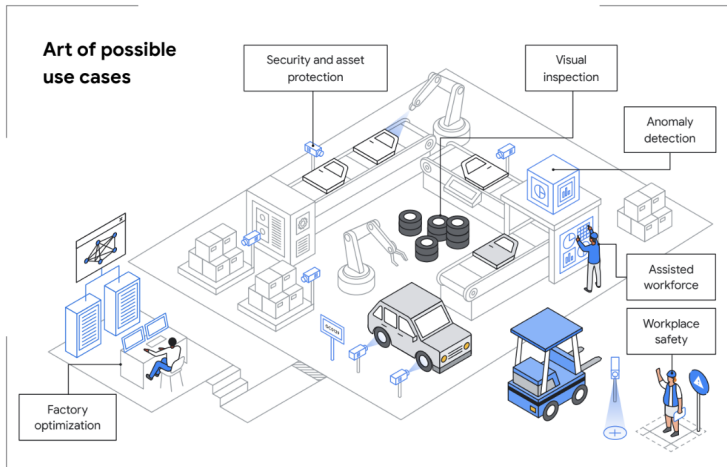


Figure: Workload Migration Across Edge, Regional, and Core Cloud.

<https://cloud.google.com/distributed-cloud>

What is a Cloud Data Center?

- Large-scale computing facility providing cloud services over the Internet.
- Supports IaaS, PaaS, and SaaS delivery models.
- Hosts compute, storage, networking, AI workloads, and analytics services.
- Designed for scalability, reliability, fault tolerance, and energy efficiency.

Major Components of Cloud Data Centers

- Compute: CPUs, GPUs, TPUs.
- Memory: DRAM, HBM.
- Storage: SSD, HDD, object storage systems.
- Networking: switches, routers, optical interconnects.
- Cooling: air cooling, liquid cooling, immersion cooling.
- Power: transformers, UPS, generators, power distribution units.

Evolution of Data Centers

- 2000–2010: Enterprise servers, low utilization, very high cooling and human cost.
- 2010–2020: Cloud era, hyperscale DCs, virtualization, improved energy efficiency.
- Post-2020: AI-driven GPU superclusters, extreme rack power density.
- Shift from CPU-centric to GPU-centric architecture.

Traditional vs AI Data Centers

- Traditional DCs: CPU workloads, 5–15 kW per rack.
- AI DCs: GPU/TPU workloads, 40–120 kW per rack.
- Massive parallel computation and memory bandwidth required.
- Cooling and power delivery become dominant engineering challenges.

Major Cost Components

- IT Processing Cost: electricity for compute, memory, storage, networking.
- Cooling Cost: electricity for heat removal systems.
- Power Delivery Losses: UPS, transformers, power distribution inefficiencies.
- Human Operations Cost: engineers, operators, security, maintenance.
- Infrastructure and Maintenance: building, networking, repairs.

Typical Cost Distribution (Modern AI Data Centers)

- IT Computation: 50–65%.
- Cooling: 15–35%.
- Power Delivery Loss: 5–10%.
- Human Operations: 3–8%.
- Infrastructure and Maintenance: 2–7%.

Large Language Models (LLMs) and Cloud Services

LLMs have fundamentally changed how cloud computing systems and data centers are designed and operated.

In traditional cloud computing, workloads such as web hosting, database management, and enterprise applications dominated infrastructure design.

However, modern AI systems require massive computational power, extremely high memory bandwidth, and distributed GPU clusters.

- What multi-billion parameter models are,
- Why they require specialized cloud infrastructure,
- How data centers are redesigned for AI workloads,
- And how AI services are delivered at scale.

What is a Parameter in an LLM?

A parameter is a numerical value learned during the training process of a neural network. These parameters consist of weights and biases.

Weights determine the strength of the connection between neurons. They encode relationships between words, patterns, and concepts.

Biases allow neurons to activate even when input signals are weak, increasing the flexibility of the model.

In a 70 billion parameter model:

$$70 \times 10^9 \text{ parameters}$$

If stored in 16-bit precision:

$$70B \times 2 \text{ bytes} \approx 140GB$$

This means the model must occupy at least 140GB of GPU memory during inference.

Why Model Size Matters

Increasing the number of parameters increases the representational capacity of the model.

Larger models can:

- Capture more complex linguistic relationships,
- Perform multi-step reasoning,
- Generalize better across tasks,
- Store larger amounts of statistical knowledge.

However, increasing model size also increases:

- Memory requirements,
- Power consumption,
- Training cost,
- Inference latency.

Thus, there is a trade-off between performance and infrastructure cost.

Transformer Architecture and Compute Demand

Modern LLMs are based on the Transformer architecture.

The transformer uses:

- Self-attention mechanisms,
- Multi-head attention layers,
- Feed-forward neural networks,
- Layer normalization.

The dominant computation is large-scale matrix multiplication.

Matrix multiplication operations are highly parallelizable, which is why GPUs and tensor accelerators are essential for AI workloads.

Cloud Computing Service Models for AI

Cloud computing enables AI deployment through different service models.

Infrastructure as a Service (IaaS) provides GPU-enabled virtual machines. Organizations rent raw compute power.

Platform as a Service (PaaS) provides managed ML platforms for training and deployment.

AI-as-a-Service (AlaaS) provides API-based access to large models without requiring infrastructure management.

This layered service model allows enterprises to choose between flexibility and convenience.

Data Center Architecture for AI Workloads

Traditional data centers were optimized for CPU-based workloads.

AI data centers are optimized for:

- GPU clusters,
- High-bandwidth networking,
- Parallel storage systems,
- Advanced cooling systems.

High-speed interconnect technologies such as NVLink and InfiniBand allow GPUs to communicate at hundreds of GB/s.

Without high-speed communication, distributed training becomes inefficient.

Training vs Inference: Economic Perspective

Training is compute-intensive and expensive. It may take weeks and cost millions of dollars.

Inference happens after training and involves generating predictions for users.

Inference must balance:

- Low latency (fast responses),
- High throughput (many users),
- Cost efficiency.

Cloud providers often separate training clusters from inference clusters.

Examples of Major Large Language Models (LLMs)

LLMs are developed by major technology companies and differ in parameter size, architecture, and capabilities.

OpenAI:

- GPT-3 (2020): 175B parameters.
- GPT-4 (2023): larger mixture-of-experts architecture (exact size undisclosed).

Google DeepMind:

- PaLM (2022): 540B parameters.
- Gemini (2023–): multimodal models optimized for reasoning and efficiency.

Meta:

- LLaMA family: 7B, 13B, 34B, and 70B parameter open-weight models.

Other AI Labs:

- Anthropic Claude: safety-focused models with long context windows.
- Mistral AI: efficient small models (e.g., 7B) and mixture-of-experts architectures.

Key idea: Modern LLM progress focuses on **better architectures, efficiency, and multimodal capability**, not only larger parameter counts.

Mixture-of-Experts (MoE) Architecture

Mixture-of-Experts (MoE) is a neural network architecture where multiple specialized sub-networks (**experts**) exist, but only a small subset is activated for each input.

How it works:

- 1 Input tokens enter the model.
- 2 A **gating network** selects the most relevant experts.
- 3 Only a few experts (typically **2–4**) process each token.
- 4 The outputs of the selected experts are combined to produce the final result.

Why MoE is important:

- Enables extremely large models with **trillions of parameters**.
- Only a fraction of parameters are active per token, reducing computation.
- Experts can specialize in different patterns or knowledge domains.

Examples: GPT-4 (estimated MoE), Switch Transformer, Mixtral.

Understanding LLM Memory Requirements

Estimating VRAM Requirements

Parameter-to-Memory Mapping: To determine if an LLM can be served on a specific hardware node, the memory footprint (M) can be calculated:

$$M_{GB} = \left(\Phi \times \frac{Q}{8} \right) \times 1.25$$

- Φ : Number of parameters (in billions).
- Q : Quantization bit-width (4, 8, 16).
- 1.25: Overhead for KV cache and activations.

Quantization bit-width determines the precision used to represent model weights and activations. Typically reducing from 32-bit floating-point (FP32) to 8-bit (INT8) or 4-bit (INT4) integers. Lower bit-widths, such as 4-bit or 2-bit, significantly reduce memory usage (up to 75%) and speed up inference, often with minimal accuracy loss.

Case Study: Google Cloud and Hugging Face

Google Cloud TPU

- TPUs are optimized specifically for matrix multiplication.
- Designed for large-scale Transformer training.
- **Interconnect:** Optical Circuit Switching enabling 3D Torus topology.

Computational Capability

Google Brain TPU v5p delivers up to **459 TFLOPS (Brain Floating Point BF16 $\approx$ FP32)** per chip, enabling trillion-parameter training (e.g., Gemini).

Hugging Face: Model-as-a-Service Abstraction

- **Hub:** Central repository for models, datasets, and demos.
- **Inference Endpoints:**
 - Kubernetes-based auto-scaling clusters.
 - Optimized via Text Generation Inference (TGI).
- **Bridging Gap:** From laptop-scale research to H100-scale deployment.

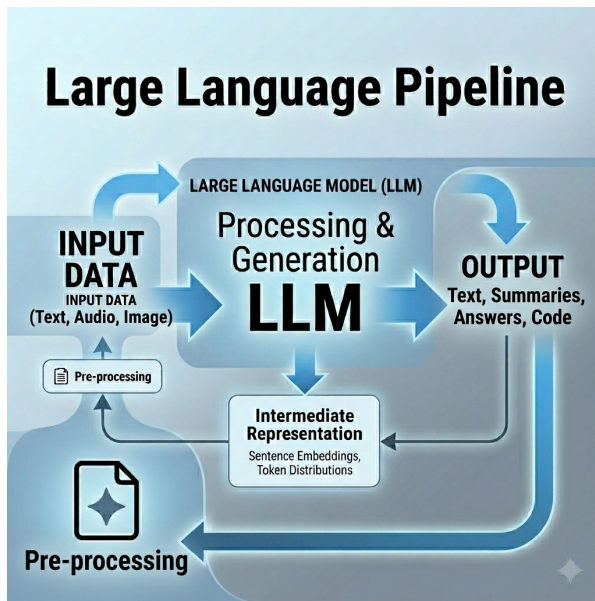
Service Delivery: Throughput vs Model Size

High Bandwidth Memory (HBM) offers massive data transfer rates, often reaching over 1 TB/s (terabytes per second) per stack to meet the demands of AI, GPUs, and high-performance computing (HPC).

- 1 TB/s+ per stack, significantly higher than DDR5 or GDDR6

$$\text{Inference Latency} \approx \frac{\Phi \times 2}{\text{HBM Bandwidth}} + \text{Network Overhead} \quad (1)$$

- Φ : Model size in billions.
- 2 bytes per parameter (FP16).
- **Optimization:** Speculative decoding reduces user-perceived latency.



AI Platforms: Training and Inference

Training Phase: For example, while optimizing weights w via Gradient Descent:

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

Hosting & Inference:

- Model Registry storage.
- High-throughput Tensor processing.
- API Endpoint deployment.

Key Insight: Training models larger than 7B parameters is generally infeasible on consumer GPUs (≤ 24 GB VRAM) without parameter-efficient fine-tuning (PEFT) methods like Low-Rank Adaptation (LoRA).

Fine-tuning 7B–8B models typically requires high-end GPUs (e.g., V100/A100). While CPU offloading can help avoid GPU memory limits by moving data to system RAM or NVMe, it usually slows performance by 10–20 $\times$.

Training vs Inference: Memory Explosion Explained

Resource Estimation for 1-Billion Models (Inference and Training)

	Inference	Training
Weights	Yes	Yes
Gradients	No	Yes
Optimizer States	No	Yes
KV Cache	Yes	Partial
Memory / 1B params	2–4 GB	16–18 GB

- **Edge:** Quantized inference and experimentation.
- **Edge Cloud:** Low-latency AI services.
- **Cloud:** Foundation model training.

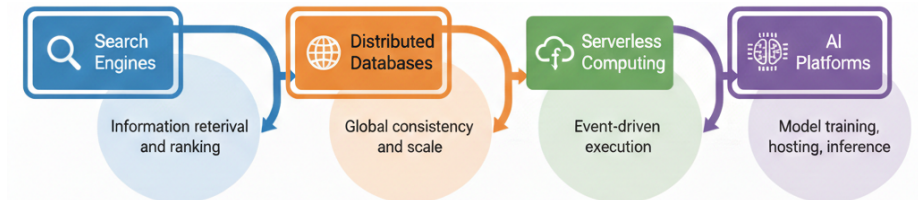
Resource Estimation for N -Billion Models (Inference)

Model Size	Precision	VRAM	Target GPU	Sys RAM
7B–8B	Q4	~6 GB	L4 / Arc 140T	16 GB
14B–20B	Q4	~12 GB	A10G / L40S	32 GB
30B–35B	Q4	~24 GB	L40S (48GB)	64 GB
70B–80B	Q4	~48 GB	A100 (80GB)	128 GB
70B+	FP16	~160 GB	2×H100	256 GB

How Search Engines Work: System Overview

- Search Engines → Information retrieval and ranking.
- Distributed Databases → Global consistency and scale.
- Serverless Computing → Event-driven execution.
- AI Platforms → Model training, hosting, inference.
- Modern search engines operate using three major stages:
 - 1 Crawling
 - 2 Indexing
 - 3 Ranking
- Designed to handle:
 - Trillions of documents
 - Billions of daily queries
 - Millisecond response times

Search Engine Architecture



- Web Crawlers → Fetch web content.
- Index Builders → Build inverted indexes.
- Ranking Models → Use ML-based scoring.

Ranking and AI in Search

- Modern ranking uses:
 - Natural Language Processing (NLP)
 - Graph Neural Networks (Web Graph)
 - Deep Ranking Models (BERT, MUM)
- **Key signals:** Relevance, Content quality, Authority, User intent
- **Insight:** Search engines are large-scale AI inference systems.

Google Cloud Spanner: Globally Distributed Database

- Google Cloud Spanner is a:
 - Mission-critical relational database service
 - Globally distributed and Strongly consistent
 - Combines traditional relational structure (schemas, SQL, ACID transactions) with non-relational horizontal scale.
- Supports:
 - SQL queries
 - ACID transactions
 - High availability across regions

Architecture: TrueTime and Global Consistency

- Core innovation: **TrueTime API** is a globally distributed clock service developed by Google to provide highly accurate, synchronized timestamps
- Combines: Atomic clocks and GPS clocks
- Enables: External consistency and Global serializability

Global Consistency $\approx$ Precise Time Synchronization

Serverless Computing: Core Concept

- **Serverless computing abstracts:**

- Infrastructure provisioning
- Resource management
- Autoscaling

- Developers write:

- Event-driven functions
- Stateless code blocks

Platforms:

- **AWS Lambda:** Event-triggered execution

- HTTP requests, Database updates, File uploads, IoT events
- Features: Millisecond billing, Automatic scaling, Zero server management

- **Google Cloud Functions:** Event Driven Architecture

- Native integration with: Firebase, BigQuery, Pub/Sub, Cloud Storage
- Execution characteristics: Stateless, Short-lived, High parallelism

Hugging Face: AI Model Ecosystem

- World's largest repository of:
 - AI models
 - Datasets
 - Evaluation benchmarks
- Provides:
 - Model hosting
 - Inference APIs
 - Training libraries
- **Hugging Face Architecture**
 - Model Hub → Centralized repository
 - Transformers Library → Training + inference
 - Accelerate → Distributed training
 - Inference Endpoints → Production deployment
- **Insight:** Hugging Face enables end-to-end AI lifecycle management.

Databricks: Unified Data + AI Platform

- Built on Apache Spark to provide Data science, AI, and SQL
- **Lakehouse Architecture:** Combines data warehouse and data lake scalability



Cloud platforms

Platform	Role	Core Strength	Use Case
Google Search	IR Engine	Ranking + NLP	Web search
Cloud Spanner	Database	Global consistency	Banking, fintech
AWS Lambda	Compute	Event processing	APIs, IoT
Cloud Functions	Compute	Native GCP apps	Backend logic
Hugging Face	AI Hub	Models + APIs	LLM services
Databricks	Data + AI	Lakehouse	Analytics, ML

Cloud-Native AI System Design Pattern

- Frontend → Web + Mobile clients
- API Gateway → Request routing
- Serverless → Business logic
- Distributed DB → Persistent state
- AI Hub → Model inference

Goal: High scalability + low latency + cost efficiency

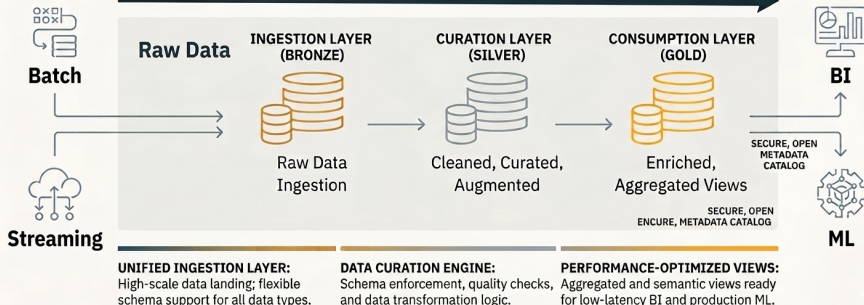
Unified System Architecture

UNIFIED SYSTEM ARCHITECTURE: DATA LAKEHOUSE PARADIGM



DATA LAKEHOUSE

Core Data Pipeline Flow: Improve Data Quality



Key Takeaways

- AI systems are now **infrastructure-scale computing platforms**.
- **Power and cooling dominate data center design**; liquid cooling is becoming essential.
- **LLM performance is memory-bandwidth bound**; GPUs with matrix engines enable scalable training and inference.
- **Cloud GPU clusters** power large-scale AI, while **edge–cloud hybrid architectures** improve cost and latency.
- Modern AI platforms rely on **distributed systems**: **serverless computing**, **AI hubs**, and **lakehouse data architectures**.
- Efficient infrastructure design determines the economic viability of **Cloud AI services**.

תודה רבה

Hebrew

Ευχαριστώ

Greek

Спасибо

Russian

Danke

German

Merci

French

Grazie

Italian

Gracias

Spanish

Obrigado

Portuguese

ধন্যবাদಗಳು

Kannada

ధన్యవాదాలు

Telugu

पूँनवाट

Punjabi

ขอบคุณ

Thai

धन्यवादः

Sanskrit

Thank You

English

धन्यवाद

Hindi & Marathi

감사합니다

Korean

நன்றி

Tamil

നന്ദി

Malayalam

આભાર

Gujarati

شكراً

Arabic

多謝

Traditional Chinese

多谢

Simplified Chinese

ありがとうございました

Japanese

<https://sites.google.com/site/animeshchaturvedi07>